

39. [25] (D. Bailey, P. Borwein, and S. Plouffe, 1996.) Explain how to compute the n th bit of the binary representation of π without knowing the previous $n - 1$ bits, by using the identity

$$\pi = \sum_{k \geq 0} \frac{1}{16^k} \left(\frac{4}{8k+1} - \frac{2}{8k+4} - \frac{1}{8k+5} - \frac{1}{8k+6} \right)$$

and doing $O(n \log n)$ arithmetic operations on $O(\log n)$ -bit integers. (Assume that the binary digits of π do not have surprisingly long stretches of consecutive 0s or 1s.)

40. [M24] Sometimes we want to divide u by v when we know that the remainder will be zero. Show that if u is a $2n$ -place number and v is an n -place number with $u \bmod v = 0$, we can save about 75% of the work of Algorithm D if we compute half of the quotient from left to right and the other half from right to left.

► **41.** [M26] Many applications of high-precision arithmetic require repeated calculations modulo a fixed n -place number w , where w is relatively prime to the base b . We can speed up such calculations by using a trick due to Peter L. Montgomery [Math. Comp. **44** (1985), 519–521], which streamlines the remaindering process by essentially working from right to left instead of left to right.

- Given $u = \pm(u_{m+n-1} \dots u_1 u_0)_b$, $w = (w_{n-1} \dots w_1 w_0)_b$, and a number w' such that $w_0 w' \bmod b = 1$, show how to compute $v = \pm(v_{n-1} \dots v_1 v_0)_b$ such that $b^m v \bmod w = u \bmod w$.
- Given n -place signed integers u, v, w with $|u|, |v| < w$, and given w' as in (a), show how to calculate an n -place integer t such that $|t| < w$ and $b^n t \equiv uv$ (modulo w).
- How do the algorithms of (a) and (b) facilitate arithmetic mod w ?

42. [HM35] Given m and b , let P_{nk} be the probability that $\lfloor (u_1 + \dots + u_m)/b^n \rfloor = k$, when $u_1, \dots, u_m$ are random n -place integers in radix b . (This is the distribution of w_n in the column addition algorithm of exercise 2.) Show that $P_{nk} = \frac{1}{m!} \langle \frac{m}{k} \rangle + O(b^{-n})$, where $\langle \frac{m}{k} \rangle$ is an Eulerian number (see Section 5.1.3).

► **43.** [22] Shades of gray or components of color values in digitized images are usually represented as 8-bit numbers u in the range $[0..255]$, denoting the fraction $u/255$. Given two such fractions $u/255$ and $v/255$, graphical algorithms often need to compute their approximate product $w/255$, where w is the nearest integer to $uv/255$. Prove that w can be obtained from the efficient formula

$$t = uv + 128, \quad w = \lfloor ([t/256] + t)/256 \rfloor.$$

*4.3.2. Modular Arithmetic

Another interesting alternative is available for doing arithmetic on large integer numbers, based on some simple principles of number theory. The idea is to have several *moduli* $m_1, m_2, \dots, m_r$ that contain no common factors, and to work indirectly with *residues* $u \bmod m_1, u \bmod m_2, \dots, u \bmod m_r$ instead of directly with the number u .

For convenience in notation throughout this section, let

$$u_1 = u \bmod m_1, \quad u_2 = u \bmod m_2, \quad \dots, \quad u_r = u \bmod m_r. \quad (1)$$

It is easy to compute $(u_1, u_2, \dots, u_r)$ from an integer number u by means of division; and it is important to note that no information is lost in this process (if

u isn't too large), since we can recompute u from $(u_1, u_2, \dots, u_r)$. For example, if $0 \leq u < v \leq 1000$, it is impossible to have $(u \bmod 7, u \bmod 11, u \bmod 13)$ equal to $(v \bmod 7, v \bmod 11, v \bmod 13)$. This is a consequence of the "Chinese remainder theorem" stated below.

We may therefore regard $(u_1, u_2, \dots, u_r)$ as a new type of internal computer representation, a "modular representation," of the integer u .

The advantages of a modular representation are that addition, subtraction, and multiplication are very simple:

$$(u_1, \dots, u_r) + (v_1, \dots, v_r) = ((u_1 + v_1) \bmod m_1, \dots, (u_r + v_r) \bmod m_r), \quad (2)$$

$$(u_1, \dots, u_r) - (v_1, \dots, v_r) = ((u_1 - v_1) \bmod m_1, \dots, (u_r - v_r) \bmod m_r), \quad (3)$$

$$(u_1, \dots, u_r) \times (v_1, \dots, v_r) = ((u_1 \times v_1) \bmod m_1, \dots, (u_r \times v_r) \bmod m_r). \quad (4)$$

To derive (4), for example, we need to show that

$$uv \bmod m_j = (u \bmod m_j)(v \bmod m_j) \bmod m_j$$

for each modulus m_j . But this is a basic fact of elementary number theory: $x \bmod m_j = y \bmod m_j$ if and only if $x \equiv y$ (modulo m_j); furthermore if $x \equiv x'$ and $y \equiv y'$, then $xy \equiv x'y'$ (modulo m_j); hence $(u \bmod m_j)(v \bmod m_j) \equiv uv$ (modulo m_j).

The main disadvantage of a modular representation is that we cannot easily test whether $(u_1, \dots, u_r)$ is greater than $(v_1, \dots, v_r)$. It is also difficult to test whether or not overflow has occurred as the result of an addition, subtraction, or multiplication, and it is even more difficult to perform division. When such operations are required frequently in conjunction with addition, subtraction, and multiplication, the use of modular arithmetic can be justified only if fast means of conversion to and from the modular representation are available. Therefore conversion between modular and positional notation is one of the principal topics of interest to us in this section.

The processes of addition, subtraction, and multiplication using (2), (3), and (4) are called residue arithmetic or *modular arithmetic*. The range of numbers that can be handled by modular arithmetic is equal to $m = m_1 m_2 \dots m_r$, the product of the moduli; and if each m_j is near our computer's word size we can deal with n -place numbers when $r \approx n$. Therefore we see that the amount of time required to add, subtract, or multiply n -place numbers using modular arithmetic is essentially proportional to n (not counting the time to convert in and out of modular representation). This is no advantage at all when addition and subtraction are considered, but it can be a considerable advantage with respect to multiplication since the conventional method of Section 4.3.1 requires an execution time proportional to n^2 .

Moreover, on a computer that allows many operations to take place simultaneously, modular arithmetic can be a significant advantage even for addition and subtraction; the operations with respect to different moduli can all be done at the same time, so we obtain a substantial increase in speed. The same kind of decrease in execution time could not be achieved by the conventional techniques

discussed in the previous section, since carry propagation must be considered. Perhaps highly parallel computers will someday make simultaneous operations commonplace, so that modular arithmetic will be of significant importance in “real-time” calculations when a quick answer to a single problem requiring high precision is needed. (With highly parallel computers, it is often preferable to run k *separate* programs simultaneously, instead of running a *single* program k times as fast, since the latter alternative is more complicated but does not utilize the machine any more efficiently. “Real-time” calculations are exceptions that make the inherent parallelism of modular arithmetic more significant.)

Now let us examine the basic fact that underlies the modular representation of numbers:

Theorem C (Chinese Remainder Theorem). *Let $m_1, m_2, \dots, m_r$ be positive integers that are relatively prime in pairs; that is,*

$$m_j \perp m_k \quad \text{when } j \neq k. \quad (5)$$

Let $m = m_1 m_2 \dots m_r$, and let $a, u_1, u_2, \dots, u_r$ be integers. Then there is exactly one integer u that satisfies the conditions

$$a \leq u < a + m, \quad \text{and} \quad u \equiv u_j \pmod{m_j} \quad \text{for } 1 \leq j \leq r. \quad (6)$$

Proof. If $u \equiv v \pmod{m_j}$ for $1 \leq j \leq r$, then $u - v$ is a multiple of m_j for all j , so (5) implies that $u - v$ is a multiple of $m = m_1 m_2 \dots m_r$. This argument shows that there is *at most* one solution of (6). To complete the proof we must now show the existence of *at least* one solution, and this can be done in two simple ways:

Method 1 (“Nonconstructive” proof). As u runs through the m distinct values $a \leq u < a + m$, the r -tuples $(u \bmod m_1, \dots, u \bmod m_r)$ must also run through m distinct values, since (6) has at most one solution. But there are exactly $m_1 m_2 \dots m_r$ possible r -tuples $(v_1, \dots, v_r)$ such that $0 \leq v_j < m_j$. Therefore each r -tuple must occur exactly once, and there must be some value of u for which $(u \bmod m_1, \dots, u \bmod m_r) = (u_1, \dots, u_r)$.

Method 2 (“Constructive” proof). We can find numbers M_j for $1 \leq j \leq r$ such that

$$M_j \equiv 1 \pmod{m_j} \quad \text{and} \quad M_j \equiv 0 \pmod{m_k} \quad \text{for } k \neq j. \quad (7)$$

This follows because (5) implies that m_j and m/m_j are relatively prime, so we may take

$$M_j = (m/m_j)^{\varphi(m_j)} \quad (8)$$

by Euler’s theorem (exercise 1.2.4–28). Now the number

$$u = a + ((u_1 M_1 + u_2 M_2 + \dots + u_r M_r - a) \bmod m) \quad (9)$$

satisfies all the conditions of (6). ■

A very special case of this theorem was stated by the Chinese mathematician Sun Tsü, who gave a rule called *tái-yen* (“great generalization”). The date of his writing is very uncertain; it is thought to be between A.D. 280 and 473. Mathematicians in mediæval India developed the techniques further, with their methods of *kuttaka* (see Section 4.5.2), but Theorem C was first stated and proved in its proper generality by Ch'in Chiu-Shao in his *Shu Shu Chiu Chang* (1247); the latter work considers also the case where the moduli might have common factors as in exercise 3. [See J. Needham, *Science and Civilisation in China* **3** (Cambridge University Press, 1959), 33–34, 119–120; Y. Li and S. Du, *Chinese Mathematics* (Oxford: Clarendon, 1987), 92–94, 105, 161–166; K. Shen, *Archive for History of Exact Sciences* **38** (1988), 285–305.] Numerous early contributions to this theory have been summarized by L. E. Dickson in his *History of the Theory of Numbers* **2** (Carnegie Inst. of Washington, 1920), 57–64.

As a consequence of Theorem C, we may use modular representation for numbers in any consecutive interval of $m = m_1 m_2 \dots m_r$ integers. For example, we could take $a = 0$ in (6), and work only with nonnegative integers u less than m . On the other hand, when addition and subtraction are being done, as well as multiplication, it is usually most convenient to assume that all of the moduli $m_1, m_2, \dots, m_r$ are odd numbers, so that $m = m_1 m_2 \dots m_r$ is odd, and to work with integers in the range

$$-\frac{m}{2} < u < \frac{m}{2}, \quad (10)$$

which is completely symmetrical about zero.

In order to perform the basic operations listed in (2), (3), and (4), we need to compute $(u_j + v_j) \bmod m_j$, $(u_j - v_j) \bmod m_j$, and $u_j v_j \bmod m_j$, when $0 \leq u_j, v_j < m_j$. If m_j is a single-precision number, it is most convenient to form $u_j v_j \bmod m_j$ by doing a multiplication and then a division operation. For addition and subtraction, the situation is a little simpler, since no division is necessary; the following formulas may conveniently be used:

$$(u_j + v_j) \bmod m_j = u_j + v_j - m_j [u_j + v_j \geq m_j]. \quad (11)$$

$$(u_j - v_j) \bmod m_j = u_j - v_j + m_j [u_j < v_j]. \quad (12)$$

(See Section 3.2.1.1.) Since we want m to be as large as possible, it is easiest to let m_1 be the largest odd number that fits in a computer word, to let m_2 be the largest odd number $< m_1$ that is relatively prime to m_1 , to let m_3 be the largest odd number $< m_2$ that is relatively prime to both m_1 and m_2 , and so on until enough m_j 's have been found to give the desired range m . Efficient ways to determine whether or not two integers are relatively prime are discussed in Section 4.5.2.

As a simple example, suppose that we have a decimal computer whose words hold only two digits, so that the word size is 100. Then the procedure described in the previous paragraph would give

$$m_1 = 99, \quad m_2 = 97, \quad m_3 = 95, \quad m_4 = 91, \quad m_5 = 89, \quad m_6 = 83, \quad (13)$$

and so on.

On binary computers it is sometimes desirable to choose the m_j in a different way, by selecting

$$m_j = 2^{e_j} - 1. \quad (14)$$

In other words, each modulus is one less than a power of 2. Such a choice of m_j often makes the basic arithmetic operations simpler, because it is relatively easy to work modulo $2^{e_j} - 1$, as in ones' complement arithmetic. When the moduli are chosen according to this strategy, it is helpful to relax the condition $0 \leq u_j < m_j$ slightly, so that we require only

$$0 \leq u_j < 2^{e_j}, \quad u_j \equiv u \pmod{2^{e_j} - 1}. \quad (15)$$

Thus, the value $u_j = m_j = 2^{e_j} - 1$ is allowed as an optional alternative to $u_j = 0$; this does not affect the validity of Theorem C, and it means we are allowing u_j to be any e_j -bit binary number. Under this assumption, the operations of addition and multiplication modulo m_j become the following:

$$u_j \oplus v_j = ((u_j + v_j) \bmod 2^{e_j}) + [u_j + v_j \geq 2^{e_j}]. \quad (16)$$

$$u_j \otimes v_j = (u_j v_j \bmod 2^{e_j}) \oplus \lfloor u_j v_j / 2^{e_j} \rfloor. \quad (17)$$

(Here $\oplus$ and $\otimes$ refer to the operations done on the individual components of $(u_1, \dots, u_r)$ and $(v_1, \dots, v_r)$ when adding or multiplying, respectively, using the convention (15).) Equation (12) is still good for subtraction, or we can use

$$u_j \ominus v_j = ((u_j - v_j) \bmod 2^{e_j}) - [u_j < v_j]. \quad (18)$$

These operations can be performed efficiently even when 2^{e_j} is larger than the computer's word size, since it is a simple matter to compute the remainder of a positive number modulo a power of 2, or to divide a number by a power of 2. In (17) we have the sum of the "upper half" and the "lower half" of the product, as discussed in exercise 3.2.1.1–8.

If moduli of the form $2^{e_j} - 1$ are to be used, we must know under what conditions the number $2^e - 1$ is relatively prime to the number $2^f - 1$. Fortunately, there is a very simple rule:

$$\gcd(2^e - 1, 2^f - 1) = 2^{\gcd(e, f)} - 1. \quad (19)$$

This formula states in particular that $2^e - 1$ and $2^f - 1$ are relatively prime if and only if e and f are relatively prime. Equation (19) follows from Euclid's algorithm and the identity

$$(2^e - 1) \bmod (2^f - 1) = 2^{e \bmod f} - 1. \quad (20)$$

(See exercise 6.) On a computer with word size 2^{32} , we could therefore choose $m_1 = 2^{32} - 1$, $m_2 = 2^{31} - 1$, $m_3 = 2^{29} - 1$, $m_4 = 2^{27} - 1$, $m_5 = 2^{25} - 1$; this would permit efficient addition, subtraction, and multiplication of integers in a range of size $m_1 m_2 m_3 m_4 m_5 > 2^{143}$.

As we have already observed, the operations of conversion to and from modular representation are very important. If we are given a number u , its modular representation $(u_1, \dots, u_r)$ may be obtained by simply dividing u by

$m_1, \dots, m_r$ and saving the remainders. A possibly more attractive procedure, if $u = (v_m v_{m-1} \dots v_0)_b$, is to evaluate the polynomial

$$(\dots (v_m b + v_{m-1})b + \dots) b + v_0$$

using modular arithmetic. When $b = 2$ and when the modulus m_j has the special form $2^{e_j} - 1$, both of these methods reduce to quite a simple procedure: Consider the binary representation of u with blocks of e_j bits grouped together,

$$u = a_t A^t + a_{t-1} A^{t-1} + \dots + a_1 A + a_0, \quad (21)$$

where $A = 2^{e_j}$ and $0 \leq a_k < 2^{e_j}$ for $0 \leq k \leq t$. Then

$$u \equiv a_t + a_{t-1} + \dots + a_1 + a_0 \pmod{2^{e_j} - 1}, \quad (22)$$

since $A \equiv 1$, so we obtain u_j by adding the e_j -bit numbers $a_t \oplus \dots \oplus a_1 \oplus a_0$, using (16). This process is similar to the familiar device of “casting out nines” that determines $u \bmod 9$ when u is expressed in the decimal system.

Conversion back from modular form to positional notation is somewhat more difficult. It is interesting in this regard to notice how the study of computation changes our viewpoint towards mathematical proofs: Theorem C tells us that the conversion from $(u_1, \dots, u_r)$ to u is possible, and two proofs are given. The first proof we considered is a classical one that relies only on very simple concepts, namely the facts that

- i) any number that is a multiple of m_1 , of m_2 , $\dots$, and of m_r , must be a multiple of $m_1 m_2 \dots m_r$ when the m_j 's are pairwise relatively prime; and
- ii) if m pigeons are put into m pigeonholes with no two pigeons in the same hole, then there must be one in each hole.

By traditional notions of mathematical aesthetics, this is no doubt the nicest proof of Theorem C; but from a computational standpoint it is completely worthless. It amounts to saying, “Try $u = a, a + 1, \dots$ until you find a value for which $u \equiv u_1 \pmod{m_1}, \dots, u \equiv u_r \pmod{m_r}$.”

The second proof of Theorem C is more explicit; it shows how to compute r new constants $M_1, \dots, M_r$, and to get the solution in terms of these constants by formula (9). This proof uses more complicated concepts (for example, Euler's theorem), but it is much more satisfactory from a computational standpoint, since the constants $M_1, \dots, M_r$ need to be determined only once. On the other hand, the determination of M_j by Eq. (8) is certainly not trivial, since the evaluation of Euler's φ -function requires, in general, the factorization of m_j into prime powers. There are much better ways to compute M_j than to use (8); in this respect we can see again the distinction between mathematical elegance and computational efficiency. But even if we find M_j by the best possible method, we're stuck with the fact that M_j is a multiple of the huge number m/m_j . Thus, (9) forces us to do a lot of high-precision calculation, and such calculation is just what we wished to avoid by modular arithmetic in the first place.

So we need an even *better* proof of Theorem C if we are going to have a really usable method of conversion from $(u_1, \dots, u_r)$ to u . Such a method was

suggested by H. L. Garner in 1958; it can be carried out using $\binom{r}{2}$ constants c_{ij} for $1 \leq i < j \leq r$, where

$$c_{ij} m_i \equiv 1 \pmod{m_j}. \quad (23)$$

These constants c_{ij} are readily computed using Euclid's algorithm, since for any given i and j Algorithm 4.5.2X will determine a and b such that $am_i + bm_j = \gcd(m_i, m_j) = 1$, and we may take $c_{ij} = a$. When the moduli have the special form $2^{e_j} - 1$, a simple method of determining c_{ij} is given in exercise 6.

Once the c_{ij} have been determined satisfying (23), we can set

$$\begin{aligned} v_1 &\leftarrow u_1 \bmod m_1, \\ v_2 &\leftarrow (u_2 - v_1) c_{12} \bmod m_2, \\ v_3 &\leftarrow ((u_3 - v_1) c_{13} - v_2) c_{23} \bmod m_3, \\ &\vdots \\ v_r &\leftarrow (\dots((u_r - v_1) c_{1r} - v_2) c_{2r} - \dots - v_{r-1}) c_{(r-1)r} \bmod m_r. \end{aligned} \quad (24)$$

Then

$$u = v_r m_{r-1} \dots m_2 m_1 + \dots + v_3 m_2 m_1 + v_2 m_1 + v_1 \quad (25)$$

is a number satisfying the conditions

$$0 \leq u < m, \quad u \equiv u_j \pmod{m_j} \quad \text{for } 1 \leq j \leq r. \quad (26)$$

(See exercise 8; another way of rewriting (24) that does not involve as many auxiliary constants is given in exercise 7.) Equation (25) is a *mixed-radix representation* of u , which can be converted to binary or decimal notation using the methods of Section 4.4. If $0 \leq u < m$ is not the desired range, an appropriate multiple of m can be added or subtracted after the conversion process.

The advantage of the computation shown in (24) is that the calculation of v_j can be done using only arithmetic mod m_j , which is already built into the modular arithmetic algorithms. Furthermore, (24) allows parallel computation: We can start with $(v_1, \dots, v_r) \leftarrow (u_1 \bmod m_1, \dots, u_r \bmod m_r)$, then at time j for $1 \leq j < r$ we simultaneously set $v_k \leftarrow (v_k - v_j) c_{jk} \bmod m_k$ for $j < k \leq r$. An alternative way to compute the mixed-radix representation, allowing similar possibilities for parallelism, has been discussed by A. S. Fraenkel, *Proc. ACM Nat. Conf.* **19** (Philadelphia: 1964), E1.4.

It is important to observe that the mixed-radix representation (25) is sufficient to compare the magnitudes of two modular numbers. For if we know that $0 \leq u < m$ and $0 \leq u' < m$, then we can tell if $u < u'$ by first doing the conversion to $(v_1, \dots, v_r)$ and $(v'_1, \dots, v'_r)$, then testing if $v_r < v'_r$, or if $v_r = v'_r$ and $v_{r-1} < v'_{r-1}$, etc., according to lexicographic order. It is not necessary to convert all the way to binary or decimal notation if we only want to know whether $(u_1, \dots, u_r)$ is less than $(u'_1, \dots, u'_r)$.

The operation of comparing two numbers, or of deciding if a modular number is negative, is intuitively very simple, so we would expect to have a much easier way to make this test than the conversion to mixed-radix form. But the following

theorem shows that there is little hope of finding a substantially better method, since the range of a modular number depends essentially on all bits of all the residues $(u_1, \dots, u_r)$:

Theorem S (Nicholas Szabó, 1961). *In terms of the notation above, assume that $m_1 < \sqrt{m}$, and let L be any value in the range*

$$m_1 \leq L \leq m - m_1. \quad (27)$$

Let g be any function such that the set $\{g(0), g(1), \dots, g(m_1 - 1)\}$ contains fewer than m_1 values. Then there are numbers u and v such that

$$g(u \bmod m_1) = g(v \bmod m_1), \quad u \bmod m_j = v \bmod m_j \quad \text{for } 2 \leq j \leq r; \quad (28)$$

$$0 \leq u < L \leq v < m. \quad (29)$$

Proof. By hypothesis, there must exist numbers $u \neq v$ satisfying (28), since g must take on the same value for two different residues. Let (u, v) be a pair of values with $0 \leq u < v < m$ satisfying (28), for which u is a minimum. Since $u' = u - m_1$ and $v' = v - m_1$ also satisfy (28), we must have $u' < 0$ by the minimality of u . Hence $u < m_1 \leq L$; and if (29) does not hold, we must have $v < L$. But $v > u$, and $v - u$ is a multiple of $m_2 \dots m_r = m/m_1$, so $v \geq v - u \geq m/m_1 > m_1$. Therefore, if (29) does not hold for (u, v) , it will be satisfied for the pair $(u'', v'') = (v - m_1, u + m - m_1)$. ■

Of course, a similar result can be proved for any m_j in place of m_1 ; and we could also replace (29) by the condition “ $a \leq u < a + L \leq v < a + m$ ” with only minor changes in the proof. Therefore Theorem S shows that many simple functions cannot be used to determine the range of a modular number.

Let us now reiterate the main points of the discussion in this section: Modular arithmetic can be a significant advantage for applications in which the predominant calculations involve exact multiplication (or raising to a power) of large integers, combined with addition and subtraction, but where there is very little need to divide or compare numbers, or to test whether intermediate results “overflow” out of range. (It is important not to forget the latter restriction; methods are available to test for overflow, as in exercise 12, but they are so complicated that they nullify the advantages of modular arithmetic.) Several applications of modular computations have been discussed by H. Takahasi and Y. Ishibashi, *Information Proc. in Japan* **1** (1961), 28–42.

An example of such an application is the exact solution of linear equations with rational coefficients. For various reasons it is desirable in this case to assume that the moduli $m_1, m_2, \dots, m_r$ are all prime numbers; the linear equations can be solved independently modulo each m_j . A detailed discussion of this procedure has been given by I. Borosh and A. S. Fraenkel [*Math. Comp.* **20** (1966), 107–112], with further improvements by A. S. Fraenkel and D. Loewenthal [*J. Res. National Bureau of Standards* **75B** (1971), 67–75]. By means of their method, the nine independent solutions of a system of 111 linear equations in 120 unknowns were obtained exactly in less than 20 minutes on a CDC 1604 computer. The same procedure is worthwhile also for solving simultaneous linear equations

with floating point coefficients, when the matrix of coefficients is ill-conditioned. The modular technique (treating the given floating point coefficients as exact rational numbers) gives a method for obtaining the *true* answers in less time than conventional methods can produce reliable *approximate* answers! [See M. T. McClellan, *JACM* **20** (1973), 563–588, for further developments of this approach; and see also E. H. Bareiss, *J. Inst. Math. and Appl.* **10** (1972), 68–104, for a discussion of its limitations.]

The published literature concerning modular arithmetic is mostly oriented towards hardware design, since the carry-free properties of modular arithmetic make it attractive from the standpoint of high-speed operation. The idea was first published by A. Svoboda and M. Valach in the Czechoslovakian journal *Stroje na Zpracování Informací* (*Information Processing Machines*) **3** (1955), 247–295; then independently by H. L. Garner [*IRE Trans.* **EC-8** (1959), 140–147]. The use of moduli of the form $2^{e_j} - 1$ was suggested by A. S. Fraenkel [*JACM* **8** (1961), 87–96], and several advantages of such moduli were demonstrated by A. Schönhage [*Computing* **1** (1966), 182–196]. See the book *Residue Arithmetic and Its Applications to Computer Technology* by N. S. Szabó and R. I. Tanaka (New York: McGraw–Hill, 1967), for additional information and a comprehensive bibliography of the subject. A Russian book published in 1968 by I. Y. Akushsky and D. I. Yuditsky includes a chapter about complex moduli [see *Rev. Roumaine de Math. Pures et Appl.* **15** (1970), 159–160].

Further discussion of modular arithmetic can be found in Section 4.3.3B.

*The notice-board had said he was in Room 423,
but the numbering system, nominally consecutive,
seemed to have been applied on a plan that could only
have been the work of a lunatic or a mathematician.*

— ROBERT BARNARD, *The Case of the Missing Brontë* (1983)

EXERCISES

1. [20] Find all integers u that satisfy all of the following conditions: $u \bmod 7 = 1$, $u \bmod 11 = 6$, $u \bmod 13 = 5$, $0 \leq u < 1000$.

2. [M20] Would Theorem C still be true if we allowed a , u_1 , u_2 , $\dots$, u_r and u to be arbitrary real numbers (not just integers)?

► 3. [M26] (*Generalized Chinese Remainder Theorem.*) Let $m_1, m_2, \dots, m_r$ be positive integers. Let m be the least common multiple of $m_1, m_2, \dots, m_r$, and let $a, u_1, u_2, \dots, u_r$ be any integers. Prove that there is exactly one integer u that satisfies the conditions

$$a \leq u < a + m, \quad u \equiv u_j \pmod{m_j}, \quad 1 \leq j \leq r,$$

provided that

$$u_i \equiv u_j \pmod{\gcd(m_i, m_j)}, \quad 1 \leq i < j \leq r;$$

and there is no such integer u when the latter condition fails to hold.

4. [20] Continue the process shown in (13); what would $m_7, m_8, m_9, \dots$ be?

► 5. [M23] (a) Suppose that the method of (13) is continued until no more m_j can be chosen. Does this “greedy” method give the largest attainable value $m_1 m_2 \dots m_r$ such